Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

THE EVOLUTION OF MACHINE LEARNING AND AI

Week 8 Core Booklet

Faster Language Models, and Your First Model

Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 8 of 12
Reading time	About 45 minutes for Chapters 1 to 3. Chapters 4 to 7 are step-by-step guides: keep them open while you do the lab at home.
Goes with	The Week 8 lecture and video · the notebook `capstone_step2.ipynb` · the Week 8 form on Tutors

What this booklet is. The required reading for Week 7, in the same order as the lecture. Chapters 1 to 3 explain the topic and your project step. Chapters 4 to 7 assume you have **never used** Google Colab, Python or Ollama, and walk you through every click.

Contents

1 Faster language models . . . 3
2 How this connects to your project . . . 8
3 This week's project step: your first model . . . 9
4 Part A at home: the speed visualiser . . . 13
5 Using this week's notebook (Part B) . . . 14
6 When it goes wrong . . . 16
7 The Week 8 form . . . 17
8 Check yourself . . . 18

YOUR WEEK, IN ORDER

1. Watch the lecture, then the walkthrough video (about 30 minutes, on Tutors).
2. Read Chapters 1 to 3 (about 45 minutes).
3. Lab at home, Part A (15 minutes): the speed visualiser. Chapter 4.
4. Lab at home, Part B (45 minutes): your first model. Chapters 5 and 6.
5. Hand in the Week 8 form before the Week 9 lecture on Monday 16 November. Chapter 7.

1 Faster language models

COVERS	Lecture Part 1
TIME	20 minutes
IN BRIEF	Attention makes long text expensive: twice the text, four times the work. Four ideas make language models faster or smarter for the money, and every one of them moves the cost somewhere else. So always measure cost as well as accuracy.

1.1 Why long text is slow

Last week you met **attention**, the heart of a transformer: when the model reads a sentence, every word looks at every other word, to decide which ones matter to it. In “the starter rose all night”, the word “rose” pays attention to “starter”, because that is what rose.

Each of those looks is one **comparison**. Six words make six times six, 36 comparisons. Twelve words make 144. Doubling the words multiplied the comparisons by four.

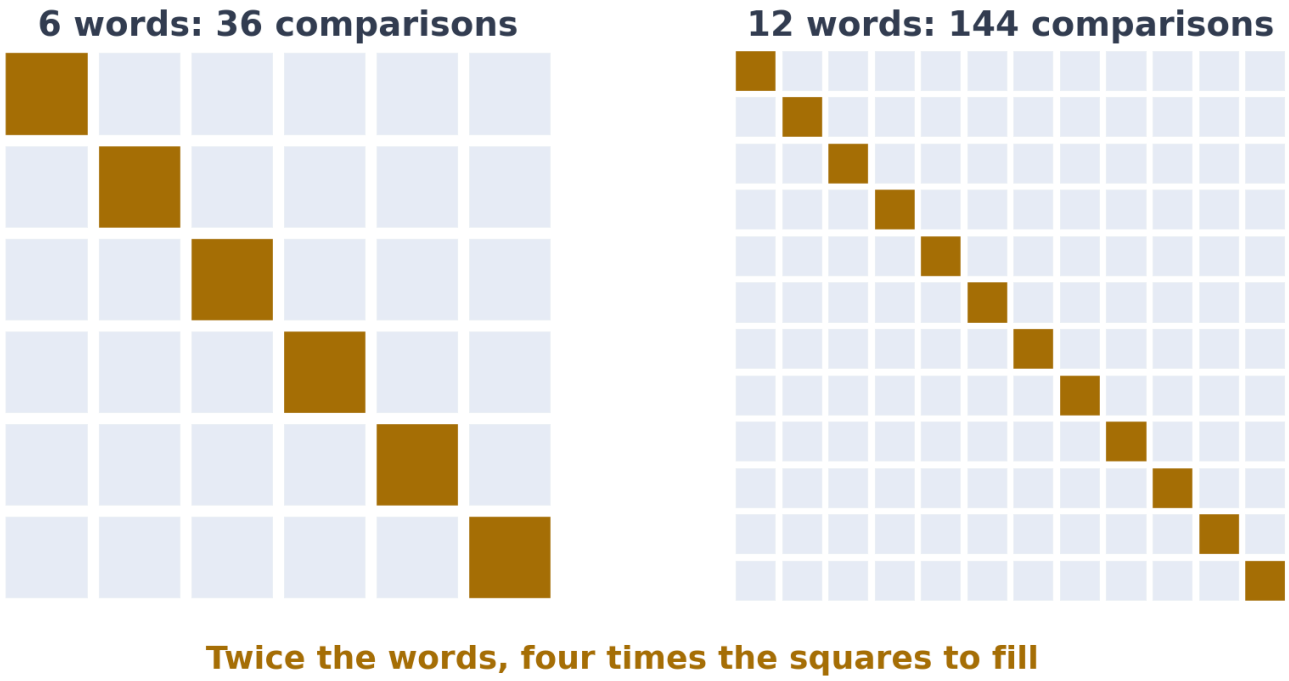


Figure 1: Each square is one comparison. Twice the words, four times the squares.

KEY IDEA

Attention compares every word with every other word. **Twice the text means four times the work**, and far more memory, because the model keeps track of the comparisons while it writes.

1.2 It gets out of hand quickly

Scale that up to real documents. One thousand words is about a million comparisons. Eight thousand words, 64 million. Thirty-two thousand words, about the length of a short book, is about a thousand million: a billion. Mathematicians call this growth **quadratic**: it grows with the square of the length.

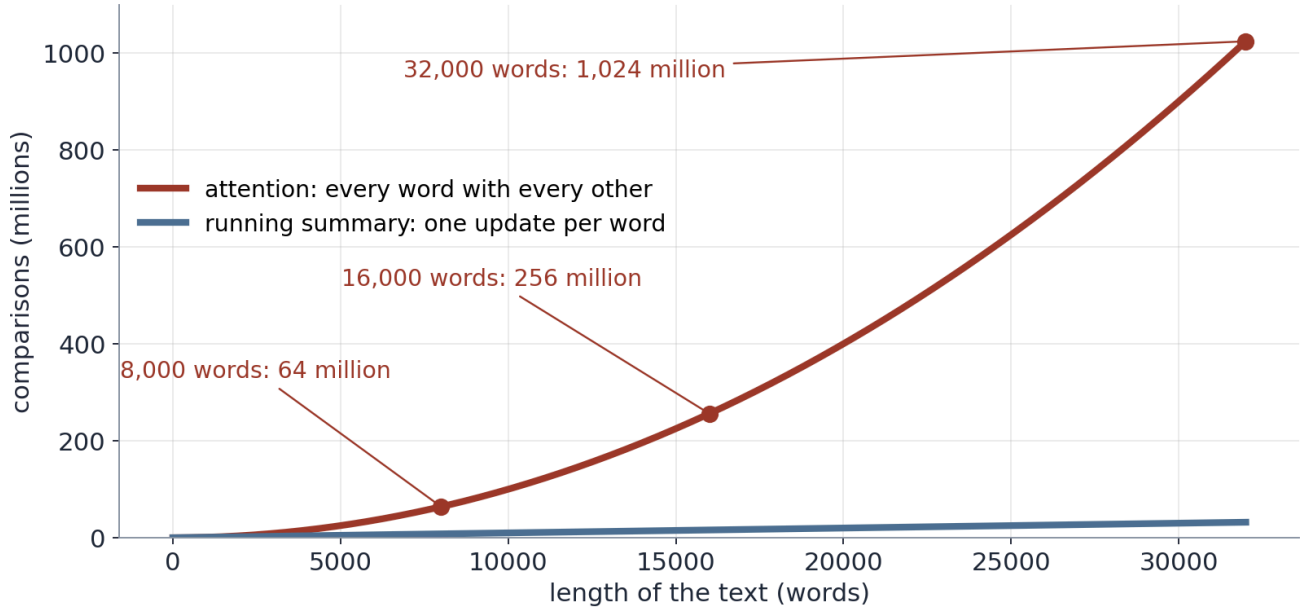


Figure 2: Attention (red) grows with the square of the length; a running summary (blue) grows in a straight line.

That is why long documents, long chat histories and large code bases are slow and expensive for today's models, and why companies charge for every token. And long text is exactly what people want: summarise a 300-page report, compare two contracts, remember a whole term's conversation, read a whole code base.

1.3 Idea 1: read with a notepad (state-space models)

A family of designs called **state-space models** reads the words in order and keeps a small running summary, like a notepad, updated at each word. It never goes back to compare the new word with every earlier one. The best known are S4 (2021) and Mamba and Mamba-2 (2023 and 2024). It is the recurrent reader from Week 4, trained in a smarter way.

Read one word, update a small notepad, move on. The notepad never gets bigger.

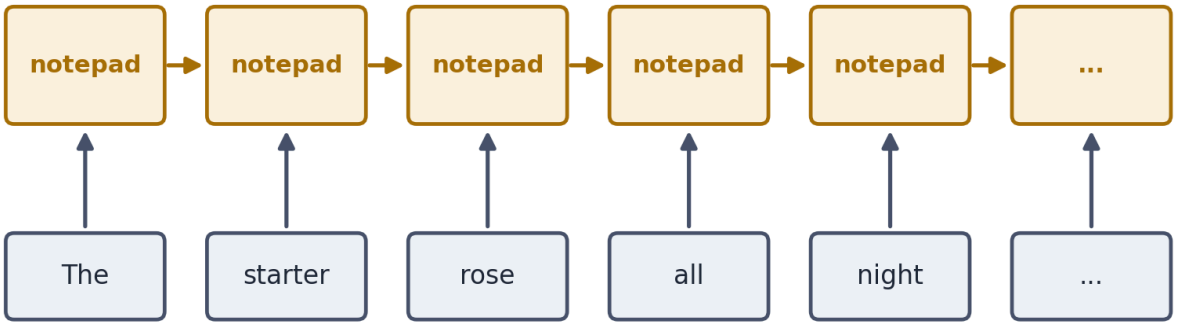


Figure 3: Read one word, update a small notepad, move on. The notepad never gets bigger.

EVERYDAY ANALOGY

Reading a novel, you do not re-read every earlier page each time you start a new sentence. You keep a few notes in your head and carry on.

Because the notepad never grows, **twice the text means only twice the work**, and the memory stays the same. You will measure this yourself in Part A.

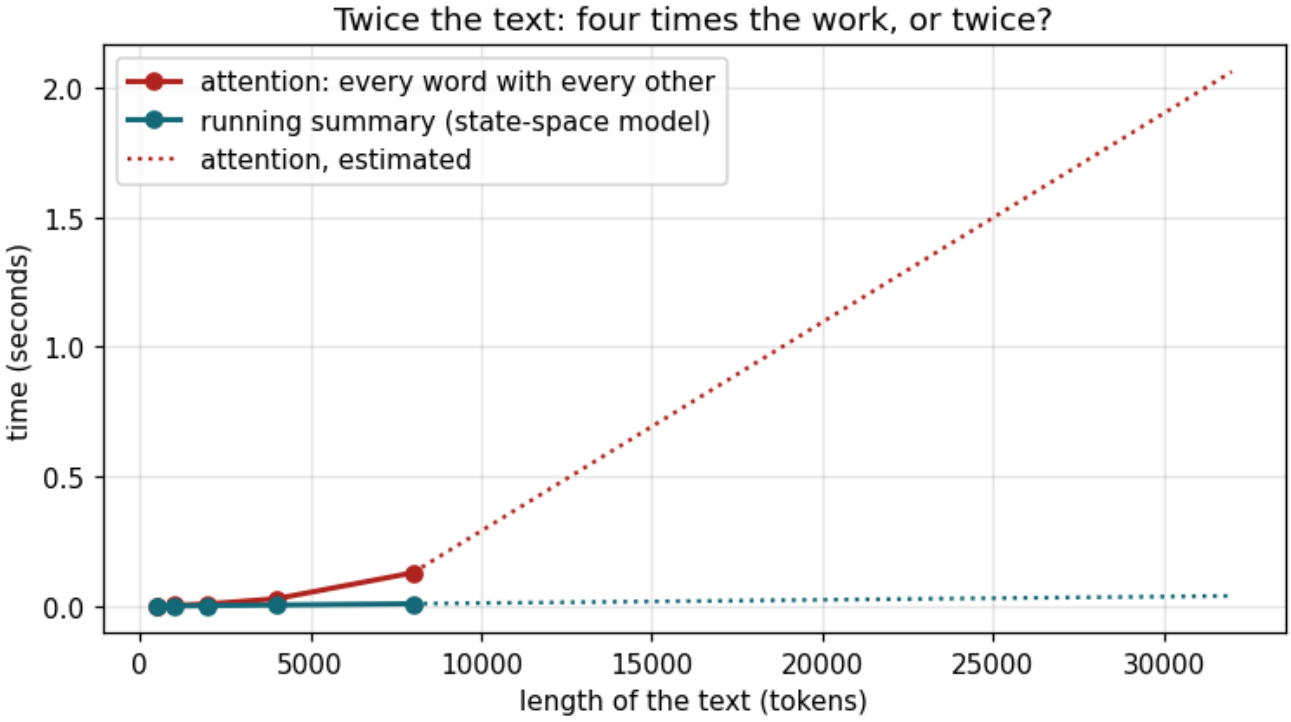


Figure 4: Measured on a laptop. Attention's time curves upwards; the running summary stays almost flat. At 32,000 tokens, attention would need about 4 GB just for its comparisons; the notepad needs 64 bytes.

1.4 What the notepad forgets

Every design is a trade-off. Attention can look back at any earlier word, **exactly**. The notepad is cheap, but small: it cannot hold every detail of a long text, so it can forget the very detail you ask about later. That is why researchers build **hybrids**: mostly cheap notepad layers, with a few attention layers to look back exactly when it matters. Jamba (2024) is one example.

Two other designs have the same aim. **RetNet** (2023) is transformer-like but can run as a reader with a memory. **RWKV**, an open community model, reads in order like a recurrent network but trains like a transformer. The shared idea: **twice the text should cost about twice as much, not four times**.

1.5 Idea 2: a team of experts (mixture of experts)

In a **mixture of experts**, the model is split into many parts called experts. For each word, a small **router** picks just a few experts to do the work; the rest rest. You get a very large model, which knows a lot, but pay to run only a small part of it for each word.

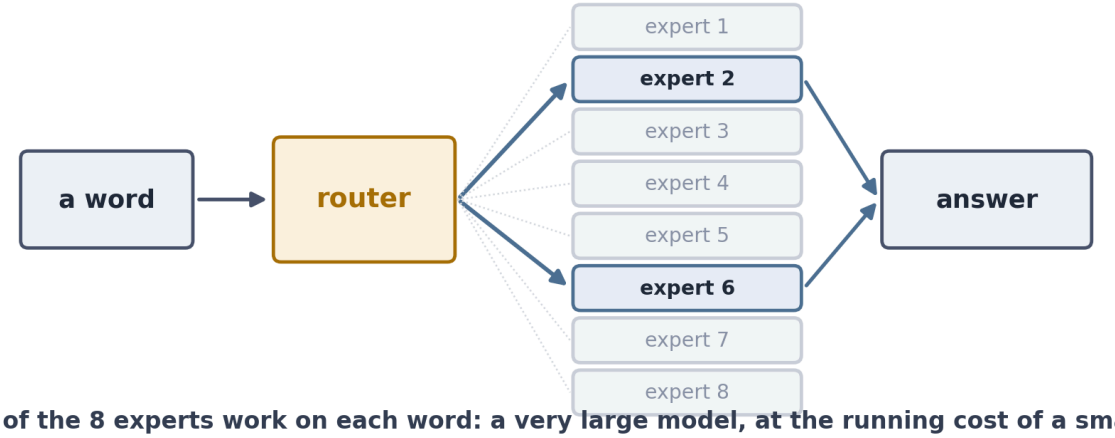


Figure 5: The router sends each word to 2 of 8 experts.

For example, Mixtral 8×7B (2023) has about 47 billion numbers in total but uses about 13 billion for each word. The catch: every expert must still be stored in memory, ready to be picked. It saves computing time, not memory.

1.6 Idea 3: think for longer (reasoning models)

A **reasoning model** writes out its thinking, step by step, before it answers, like a student showing their working. OpenAI's o1 (2024) and DeepSeek-R1 (2025) are examples. Each answer costs more, because every thinking token is still a token. But harder questions in maths, code and planning get solved that a quick answer would get wrong.

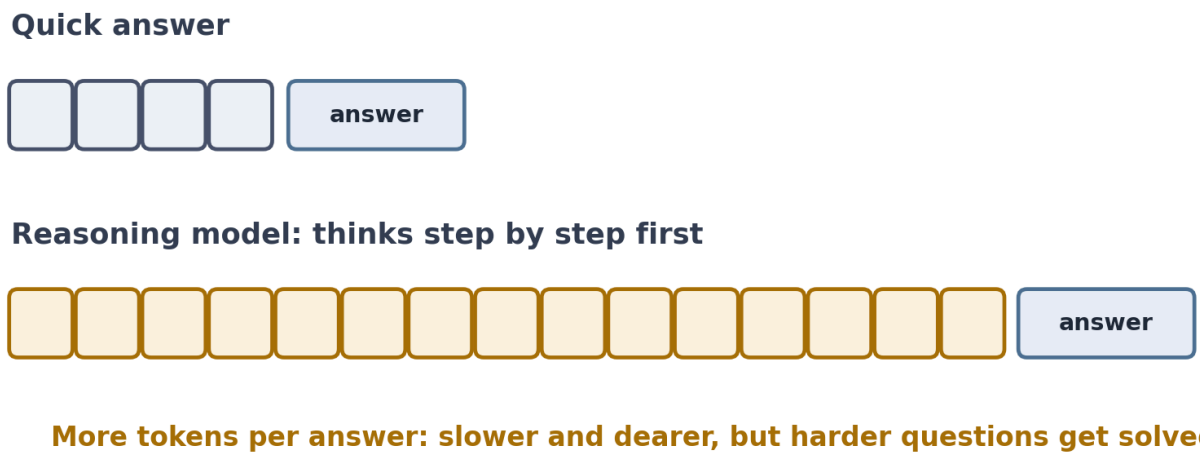


Figure 6: More tokens per answer: slower and dearer, but better on hard questions.

1.7 Four ideas, side by side

Idea	Saves	Costs
Notepad (state-space)	Time and memory on long text	Can forget exact details
Other attention-free designs	Time and memory on long text	Newer, less tested
Mixture of experts	Computing time per word	Memory: all experts stored
Reasoning models	Wrong answers to hard questions	Time and money per answer

KEY IDEA

Nothing is free: every idea moves the cost somewhere else. **Always measure cost (time, size, memory) as well as accuracy.** A model that is a little less accurate but ten times cheaper can be the better choice, and you only know if you measure both.

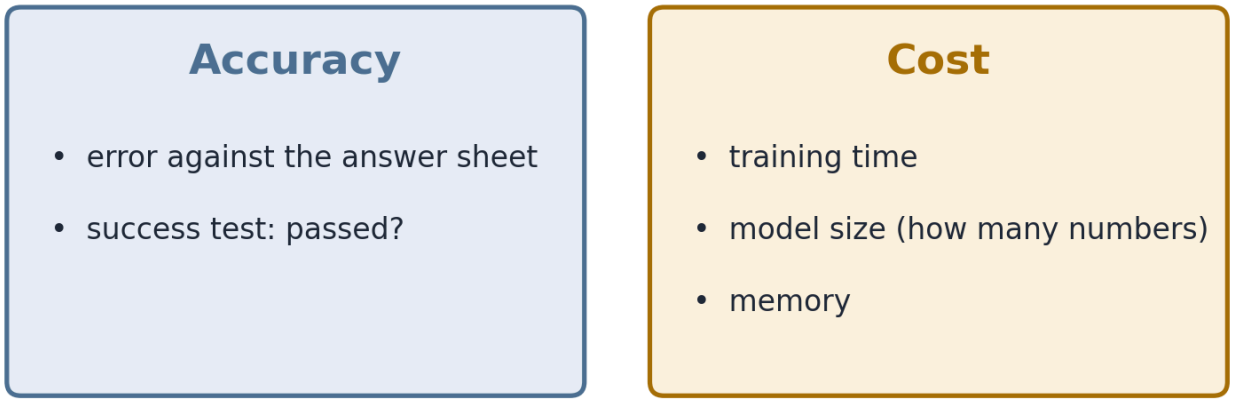


Figure 7: The two things to measure, from this week on.

2 How this connects to your project

COVERS Lecture Part 2
TIME 5 minutes
IN BRIEF From this week, record your model's cost as well as its accuracy. Week 9's race needs both.

When you train your first model this week, the notebook prints two kinds of numbers. **Accuracy**: your error against the answer sheet, and whether your success test passes. **Cost**: how long training took, and your model's **size**, meaning how many numbers it has inside.

Write both down. In Week 9 you race your model against an ordinary model that learns from examples, on accuracy, speed and size, and you need this week's numbers for the race.

> **WHY THIS MATTERS**
>
> Your model has about a thousand numbers inside it. A large language model has billions. That difference, and what it buys you, is the story of your final report.

3 This week's project step: your first model

COVERS	Lecture Part 2 and the video
TIME	20 minutes
IN BRIEF	Train a model that learns your law with no data, check it against your answer sheet, and run a success test you set in advance.

3.1 How it learns with no data

The same way as in Week 5. The model guesses a curve; the computer checks the guess against your law at many moments in time; it works out which of the model's numbers caused the mismatch; and it nudges them. Then it repeats, about 3,000 times, in a few seconds.

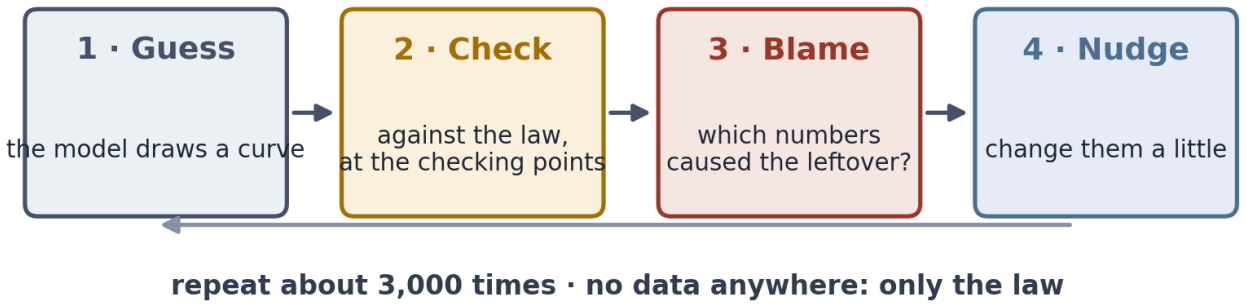


Figure 8: Guess, check, blame, nudge. The only teacher is the law.

3.2 Checking points and the leftover

The moments where the law is checked are the **checking points**: 200 of them by default. At each one, the computer measures the **leftover**, Week 5's word: how far the guess's rate of change is from what the law says. Training shrinks the leftover at every checking point at once, and the guess moves onto the right answer.

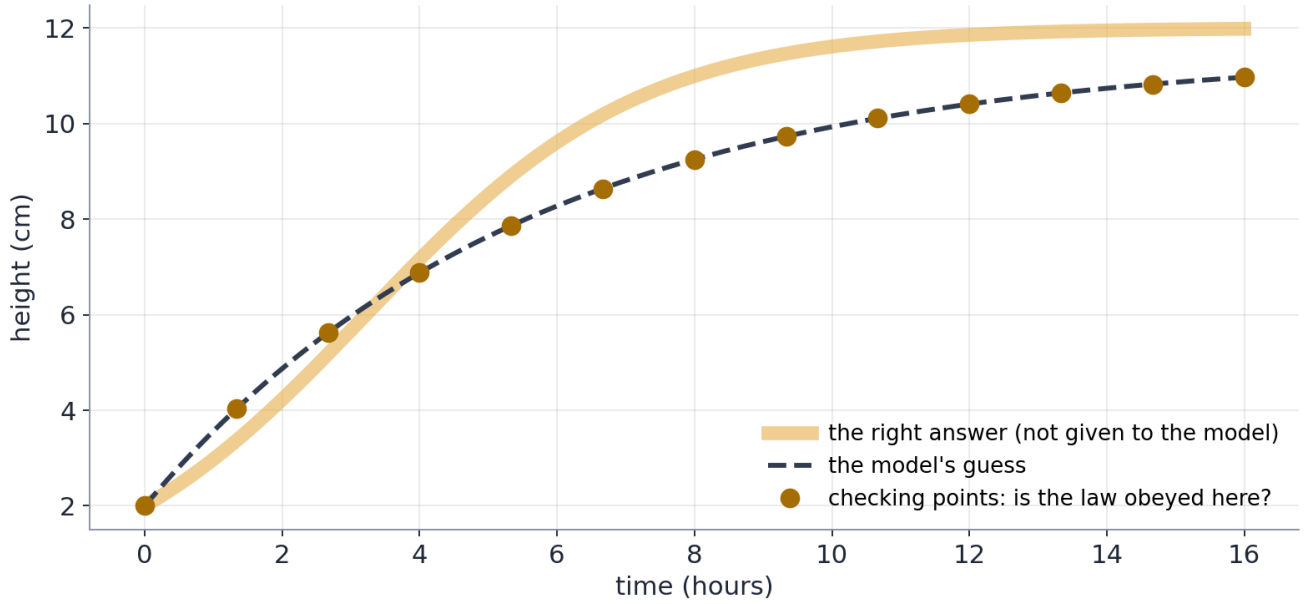


Figure 9: The guess (dashed) is checked at each orange dot. The right answer (thick) is never shown to the model.

3.3 The start is built in, and the model's shape

The start is built in (the **fade-in** from Week 5): at time zero the model is exactly your starting value, so it only learns what happens next. In this week's notebook the starting **slope** is built in too, which makes training more reliable.

The shape of your model follows the shape of your law: the same three shapes, and the same notebook parts, as last week.

Your law	The model	Notebook
One thing changes over time	One input (time), one output	Part 2
Two or three things change each other	One input (time), one output per thing	Part 3
It changes over time and along a line	Two inputs (place and time), one output	Part 4

3.4 Your success test, decided first

Before you train, decide what **good enough** means, as a number: at a time you choose, the model is within some amount of the answer sheet. For the class example: **the height at 8 hours is within 0.1 cm of the answer sheet**. Your roadmap on Moodle suggests one for your law. Deciding first stops you moving the goalposts after you see the result.

3.5 Reading the loss curve

The **loss** is the average leftover. The notebook draws it as training goes on. Three shapes to recognise:

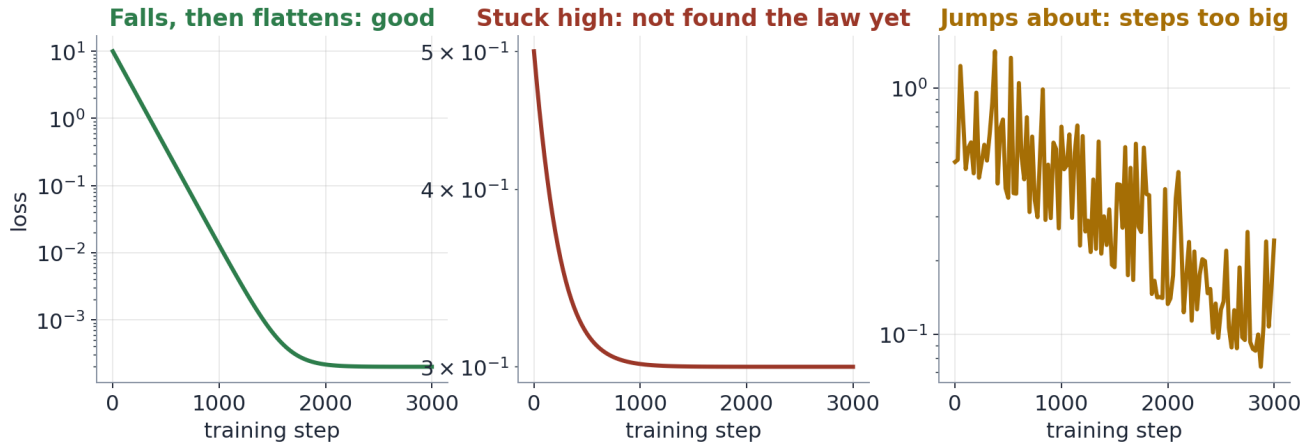


Figure 10: Falls then flattens: good. Stuck high: the law is not found yet. Jumps about: the steps are too big.

In the class example the curve is bumpy at first, because the notebook teaches the model the early hours first, then the later ones. What matters is the end: it falls, then flattens.

3.6 Break it on purpose, and the four fixes

With only **3** checking points, the class example's loss falls almost to zero, yet the model misses the answer sheet: 1.27 cm off at 8 hours, where 0.1 is allowed. The law was obeyed at three points and nowhere else.

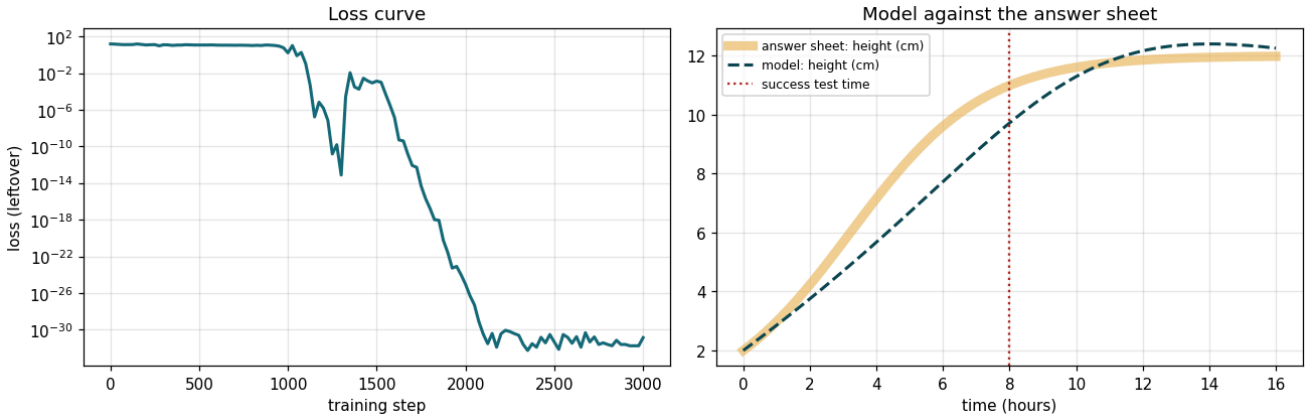


Figure 11: A tiny loss is not enough: the loss only covers the points you checked.

> **COMMON MISTAKE**
>
> A small loss only means the law is obeyed **at the checking points**. That is why your success test compares with the answer sheet, and why in Week 10 you will test at new points.

If your model fails its success test, try these **one at a time, in order**:

1. **Keep the numbers near 1**: choose sensible units (hours, not seconds).
2. **Use more checking points** (the default is 200).
3. **Use a shorter time range**: learn less time first.
4. **Take smaller steps**: a smaller learning rate, for example `lr=0.003`.

With 200 checking points again, the class example passes: 0.0007 cm off at 8 hours.

3.7 The applied checkpoint

On the form, write two or three sentences on **where your model could go wrong**: what your law leaves out (your model cannot know it, so you must say it); the numbers you chose, where they came from and how sure you are;

and Week 5′s lesson, that a model trained on a wrong law is confidently wrong. It is formative feedback, and it becomes part of section 6 of your final report.

4 Part A at home: the speed visualiser

COVERS	Lab at home, Part A
TIME	15 minutes
IN BRIEF	Time two ways of reading a text as it gets longer, and see “four times” against “twice” for yourself.

1. Open `capstone_step2.ipynb` in Google Colab (Chapter 5, steps 1 to 3) and run **Step 0**. Wait for **Ready**.
2. Run the **Part A** cell. It takes a few seconds.
3. Look at the picture. The red line is attention; the teal line is the running summary, the notepad. Dotted lines are estimates for 32,000 tokens.
4. Read the three printed lines. For 32,000 tokens, attention makes 1,024,000,000 comparisons and needs about 4.1 GB; the notepad makes 32,000 updates and needs 64 bytes.
5. On the form, answer in one sentence: **which design is faster for long text, and why?**

Your times will differ from the booklet’s, because computers differ. The **shapes** will be the same.

5 Using this week's notebook (Part B)

COVERS	Lab at home, Part B
TIME	45 minutes
IN BRIEF	Run the class example, break it and fix it, then train your first model in your part of the notebook, and download its picture.

5.1 Step by step

1. **Download** the Week 8 Lab File from Tutors and unzip it (right-click → Extract All on Windows; double-click on a Mac).
2. **Open it in Colab**: go to colab.research.google.com, click **Upload**, then **Browse**, and choose `capstone_step2.ipynb`.
3. **Save your copy**: File → Save a copy in Drive.
4. **Run Step 0** (Shift + Enter). If a warning says the notebook was not written by Google, click **Run anyway**. Wait for **Ready**.
5. **Run Part A** (Chapter 4).
6. **Run the class example**: the cell with the numbers, then the training cell. In about 5 seconds you see the loss curve and the model against the answer sheet, and these lines:

```
Biggest error against the answer sheet: 0.004907
Success test: at 8 hours, the model says 10.99 and the answer sheet says 10.99.
  Difference 0.000681, allowed 0.1: PASSED
Training time: 2 seconds. Model size: 1153 numbers. Checking points: 200.
```

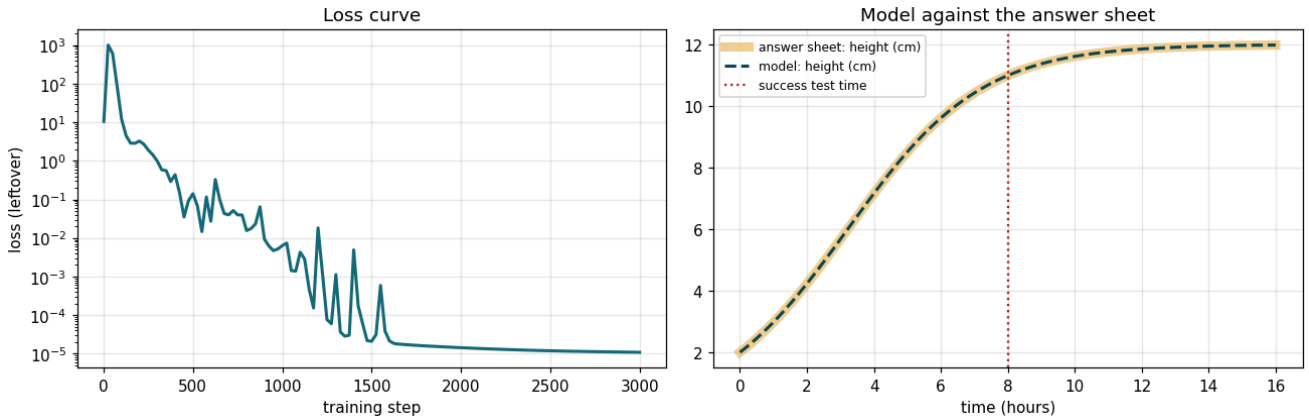


Figure 12: The class example: the loss falls and flattens, and the model (dashed) lies on the answer sheet.

7. **Break it, then fix it**: run the two cells below the class example (3 checking points, then 200).
8. **Your part**: go to Part 2, 3 or 4 (your roadmap says which) and copy in your Week 7 lines (next section).
9. **Set your success test** (`check_time` and `check_tolerance`) **before** you train.
10. **Run** your cell, then the training cell below it. Read the printed lines: error, success test, training time, model size.
11. **Download the picture**: folder icon on the left → `my_first_model.png` → ⋮ → Download.

5.2 Copying your Week 7 lines

Part	What to copy, and what is new
Part 2	Your name, time unit, numbers, start, `t_end`, law line and exact formula. **New:** `check_time`, `check_tolerance`. If your law has a square root, write `np.sqrt(np.abs(y) + 1e-9)`. Write the exact formula with `onp.` (ordinary NumPy), as in the placeholder.
Part 3	Your names, numbers, starts, `t_end3`, time unit and `my_rates`. **New:** `which` (which thing the success test checks: 0 for the first), `check_time3`, `check_tolerance3`.
Part 4	Your numbers and `L`, `t_end4`. Your law line: Week 7′s `bend` is called `u_xx` here, and `u_x` is how steep the line is. The starting values are now a small function, `u0(x)`. **New:** `check_place`, `check_time4`, `check_tolerance4`.

6 When it goes wrong

COVERS Lab at home
TIME as needed
IN BRIEF Most problems have one of five causes.

What you see	What to do
The loss stays high, and the success test says NOT YET	Try the four fixes in order, one at a time (Chapter 3).
The loss jumps up and down	The steps are too big: add `lr=0.003` inside `learn_law( ... )`.
The loss is tiny, but the model misses the answer sheet	Too few checking points: use 200 or more.
The loss says `nan`	A square root or logarithm of a negative number. Use `np.sqrt(np.abs(y) + 1e-9)`.
“Training got stuck… starting again”	Normal: the tools try a fresh start, up to three times.
Red text under a cell	Read the last line. Usually a typo, a missing bracket, or a cell you skipped: run from the top.

If you are stuck for 40 minutes on anything, stop, and post in the channel with a screenshot.

7 The Week 8 form

COVERS	Lab at home
TIME	5 minutes
IN BRIEF	Due before the Week 9 lecture, on Monday 16 November.

- **Part A**: which design is faster for long text, and why, in one sentence.
- Your **error**, and your **success test**: passed, or not yet.
- Your **training time** and **model size**.
- The picture `my_first_model.png`.
- **One problem you fixed**, and how.
- **Where your model could go wrong**: two or three sentences (the applied checkpoint).

Keep your AI-use log going: what you asked an AI tool this week, what you used, and what you changed.

8 Check yourself

COVERS Revision
TIME 10 minutes
IN BRIEF Cover the right-hand column, answer, uncover.

#	Question	Answer
1	Why is long text slow for a transformer?	Attention compares every word with every other word: twice the text, four times the work.
2	How many comparisons do 1,000 words need with attention?	About a million.
3	What does a state-space model keep as it reads?	A small running summary (a notepad) that never grows.
4	What can a small notepad not do well?	Recall exact details from far back in a long text.
5	What is a hybrid model?	Mostly notepad layers, with a few attention layers.
6	In a mixture of experts, what does the router do?	Picks a few experts to work on each word.
7	What does a mixture of experts save, and what does it not?	It saves computing time; it does not save memory.
8	Why does a reasoning model cost more per answer?	It writes out many thinking tokens before answering.
9	What is the lesson to keep?	Measure cost (time, size, memory) as well as accuracy.
10	What teaches your model, if not data?	The law, checked at the checking points.
11	Why decide your success test before training?	So you cannot move the goalposts after seeing the result.
12	The loss is tiny but the model is wrong. Why?	Too few checking points: the law is obeyed only there.
13	Name the four fixes, in order.	Numbers near 1; more checking points; shorter time range; smaller steps.